

Producto 5:

Transformación de un

archivo XML mediante

HTML, CSS y JavaScript.

Autor: Pol Valle

Año: 2026

Índice

Contenido

Introducción	3
Implementación	3
Estructura de Ficheros	3
Funcionamiento de la página web	4
HTML	4
CSS	4
JS	5
Resultados	7

Introducción

En este producto se parte del archivo XML generado en el Producto 4 (red.xml), que contiene la configuración del adaptador de red (IP, máscara, puerta de enlace, DNS primario, velocidad media de ping y trazado de ruta) y se construye en una pequeña aplicación web compuesta por un documento HTML, una hoja de estilos CSS y un script de JS que al pulsar un botón, carga el XML, lo parecía en el navegador y muestra su contenido en tablas.

Implementación

Estructura de Ficheros

El proyecto se compone de cuatro archivos con responsabilidades claramente separadas:

Archivo	Responsabilidades
index.html	Define la estructura de la página: cabecera, descripción, los dos botones de carga del XML y el contenedor donde se inyectarán las tablas con los datos.
style.css	Hoja de estilos que maqueta el contenido como tarjetas y tablas: tipografía, colores, cabeceras de tabla resaltadas,
app.js	Registra los listeners de los dos botones, descarga red.xml con fetch o lee el archivo seleccionado con FileReader, parsea el XML con DOMParser y construye dinámicamente las tablas HTML con los datos.
red.xml	Archivo XML generado en el Producto 4 con la configuración del adaptador de red. Es la fuente de datos que consume la página.

La separación entre los tres lenguajes hace que cada uno cumpla únicamente su papel: el HTML aporta la estructura de la página, el css los estilos y el JS la lógica de la cara de datos.

Funcionamiento de la página web

HTML

El documento index.html define la estructura semántica de la web, tiene cabecera con título y subtítulo, una sección de introducción donde están los controles (botón para cargar red.xml y botón secundario que abre un selector de ficheros como alternativa), un mensaje donde se ve el estado para accesibilidad y un contenedor oculto donde JS inyecta el resultado tras pulsar el botón

```
You, last week | 1 author (You)
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Configuración del dispositivo de red</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <header>
    <h1>Configuración del dispositivo de red</h1>
    <p>Producto 5 - Transformación de XML con CSS y JavaScript</p>
  </header>

  <main>
    <section class="introduccion">
      <h2>Acerca de esta página</h2>
      <p>
        Esta página muestra la configuración del dispositivo de red almacenada en el
        archivo <strong>red.xml</strong> generado en el Producto 4. Al pulsar el botón,
        JavaScript carga el XML y lo muestra formateado como tablas con la hoja de
        estilos <strong>styles.css</strong>.
      </p>

      <button id="btn-cargar">Cargar configuración del adaptador</button>
      <button id="btn-archivo">Seleccionar archivo XML ... </button>
      <input type="file" id="input-archivo" accept=".xml" hidden>

      <p id="mensaje-estado" class="estado"></p>
    </section>

    <section id="resultado" class="resultado oculto"></section>
  </main>

  <footer>
    <p>Pol Valle Montes - FP.440 Programa de comunicación personalizado</p>
  </footer>

  <script src="app.js"></script>
</body>
</html>
```

CSS

La hoja de style.css es lo que le da la forma visual (colores, posición de elementos, efectos...) a la web.

Una @media query adapta la página a pantallas pequeñas.

```

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: Arial, sans-serif;
  background-color: #eef3fb;
  color: #1f2d3d;
  padding: 20px;
  line-height: 1.5;
}

header {
  background-color: #1f3b73;
  color: white;
  padding: 20px;
  border-radius: 8px;
  margin-bottom: 20px;
}

header h1 {
  margin-bottom: 5px;
}

main {
  max-width: 900px;
  margin: 0 auto;
}

.introduccion,
.resultado {
  background-color: white;
  border-radius: 8px;
  padding: 20px;
  margin-bottom: 20px;
  box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);
}

```

JS

EL fichero de [app.js](#) se carga al final del body del html y registra los listeners sobre los 2 botones.

El botón principal lanza una función asíncrona que descarga red.xml con un fetch y pasa el texto a la función mostrar().

El botón secundario dispara el click del input file oculto, cuyo evento change lee el archivo con FileReader y llama también a mostrar().

```

...
const btnCargar = document.getElementById('btn-cargar');
const btnArchivo = document.getElementById('btn-archivo');
const inputArchivo = document.getElementById('input-archivo');
const mensaje = document.getElementById('mensaje-estado');
const resultado = document.getElementById('resultado');

// Boton principal: carga red.xml desde el servidor
btnCargar.addEventListener('click', async function () {
  try {
    const respuesta = await fetch('red.xml');
    if (!respuesta.ok) {
      throw new Error('No se ha podido leer red.xml');
    }
    const texto = await respuesta.text();
    mostrar(texto, 'red.xml');
  } catch (err) {
    ponerMensaje('Error al cargar red.xml: ' + err.message, 'error');
  }
});

// Boton secundario: abre el selector de archivos del navegador
btnArchivo.addEventListener('click', function () {
  inputArchivo.click();
});

inputArchivo.addEventListener('change', function (e) {
  const archivo = e.target.files[0];
  if (!archivo) return;

  const lector = new FileReader();
  lector.onload = function () {
    mostrar(lector.result, archivo.name);
  };
  lector.readAsText(archivo);
});

// Parsea el XML y construye las tablas
function mostrar(textoXml, origen) {
  const parser = new DOMParser();
  const doc = parser.parseFromString(textoXml, 'application/xml');

  // DOMParser no lanza error: inserta <parsererror> si el XML es invalido
  if (doc.querySelector('parsererror')) {
    ponerMensaje('El XML no esta bien formado', 'error');
    return;
  }

  const adaptador = doc.querySelector('adaptador');
  const nombre = adaptador.getAttribute('nombre');
  const ip = adaptador.querySelector('ip').textContent;
  const mascara = adaptador.querySelector('mascara').textContent;
  const puerta = adaptador.querySelector('puerta_enlace').textContent;
  const dns = adaptador.querySelector('dns_primario').textContent;
  const velocidadEl = adaptador.querySelector('velocidad_dns');
  const velocidad = velocidadEl.textContent + '!' + velocidadEl.getAttribute('unidad');

```

La función **mostrar()** valida que el XML esté bien formado, comprobando si DOMParser ha insertado un nodo `<parsererror>`.

Si lo encuentra, muestra un mensaje y aborta.

En caso de que no haya errores, extrae los campos del adaptador con el `querySelector` y construye las dos tablas con los template literals, inyectando el HTML directamente en la sección de resultado.

Resultados

Al abrir la página se muestra la cabecera con el título y los dos controles de carga.

La zona inferior aparece vacía hasta que el usuario pulsa el botón.

Configuración del dispositivo de red

Producto 5 - Transformación de XML con CSS y JavaScript

Acerca de esta página

Esta página muestra la configuración del dispositivo de red almacenada en el archivo **red.xml** generado en el Producto 4. Al pulsar el botón, JavaScript carga el XML y lo muestra formateado como tablas con la hoja de estilos **styles.css**.

Cargar configuración del adaptadorSeleccionar archivo XML...

Pol Valle Montes - FP.440 Programa de comunicación personalizado

Al pulsar Cargar configuración del adaptador, JavaScript descarga red.xml, lo parsea y lo transforma en dos tablas: la primera con los datos del adaptador (IP, máscara, puerta de enlace, DNS primario y velocidad media del DNS) y la segunda con el trazado de ruta (número de salto e IP).

← ↻ 127.0.0.1:5500/Producto-5/ Resumir ☆ ⚙ ⌵ 🤖 ...

📁 Plataformas 📁 UOC 📁 ★ Randstad 📁 Profesional 📁 📍 📁 📺 📁 18 Google Calendar ~... 📁 Mis Asignados >

Configuración del dispositivo de red

Producto 5 - Transformación de XML con CSS y JavaScript

Acerca de esta página

Esta página muestra la configuración del dispositivo de red almacenada en el archivo **red.xml** generado en el Producto 4. Al pulsar el botón, JavaScript carga el XML y lo muestra formateado como tablas con la hoja de estilos **styles.css**.

Cargar configuración del adaptadorSeleccionar archivo XML...

Configuración cargada desde "red.xml"

Configuración del adaptador *Ethernet*

Datos de red

Campo	Valor
Dirección IP	192.168.0.20
Máscara de subred	255.255.255.0
Puerta de enlace	192.168.0.1
DNS primario	192.168.0.1
Velocidad media DNS	0.0 ms

Trazado hasta el DNS (1 saltos)

Salto	Dirección IP
1	192.168.0.1

Pol Valle Montes - FP.440 Programa de comunicación personalizado